

INSTITUTO FEDERAL CATARINENSE

CAMPUS CONCÓRDIA

Curso Técnico em Informática

THE INFINITE LOOP

Jogo de Sobrevivência em RPG de Texto

Ramon Petry

Davi Patzlaff

Concórdia – SC

2026

Ramon Petry

Davi Patzlaff

THE INFINITE LOOP

Jogo de Sobrevivência em RPG de Texto

Trabalho apresentado à disciplina de Programação do Curso Técnico em Informática do Instituto Federal Catarinense Campus Concórdia, como requisito avaliativo do trimestre.

Concórdia – SC

2026

INTRODUÇÃO

Foi desenvolvido, em Python, um jogo de sobrevivência intitulado The Infinite Loop. Trata-se de um RPG de texto no qual o jogador precisa imaginar as cenas e interagir com o ambiente por meio de comandos digitados no terminal. A proposta consiste em cinquenta e uma fases textuais, ao longo das quais o jogador deve tomar as decisões corretas para alcançar a fase final e vencer o jogo.

O jogo possui uma história original, escrita pelos autores. A escolha pelo gênero de sobrevivência ocorreu com o objetivo de propor um desafio maior, uma vez que se tratava da opção mais complexa entre as três propostas apresentadas pelo professor.

2 OBJETIVOS

2.1 Objetivo Geral

Criar um jogo complexo e funcional, capaz de proporcionar imersão ao jogador, levando-o a imaginar a história, os ambientes, os monstros e os NPCs, de modo a gerar uma experiência única e pessoal para cada usuário.

2.2 Objetivos Específicos

Aprimorar as habilidades em Python desenvolvidas ao longo do trimestre; exercitar o raciocínio lógico de programação; e praticar a criação de textos narrativos aliada à implementação de mecânicas de jogo.

3 TECNOLOGIAS UTILIZADAS

Conforme orientação do professor, todo o desenvolvimento foi realizado exclusivamente em Python. Foram utilizadas apenas bibliotecas nativas da linguagem (os, io, time, random, platform e contextlib), sem dependências externas. Embora tenha sido considerada a possibilidade de dividir o código em múltiplos arquivos, optou-se por mantê-lo em um único arquivo, devido à falta de conhecimento, no momento do desenvolvimento, sobre organização modular de projetos.

4 DESENVOLVIMENTO DO JOGO

4.1 Inspiração e Conceito

O projeto teve como principal inspiração jogos clássicos de RPG de texto, como Zork, que motivaram a criação de mecânicas semelhantes. O jogo funciona por meio de comandos digitados pelo jogador, que controlam tanto o menu principal quanto as ações realizadas durante a partida.

4.2 Comandos Disponíveis

Os principais comandos disponíveis no menu principal e durante a exploração são:

- /start – Inicia a criação de personagem e o jogo;
- /sair – Fecha o programa;
- /help – Exibe a lista completa de comandos;
- /devs – Mostra informações sobre os desenvolvedores;
- /renick – Permite trocar o nome de usuário;
- /clear – Limpa o terminal;
- /tabraca – Exibe a tabela de raças, seus atributos e bônus passivos;
- /inv – Abre o inventário do jogador;
- /sts – Exibe os status atuais do personagem.

Durante o combate, o jogador conta com um submenu próprio, com as opções de atacar, fugir, usar item, consultar o inventário e consultar os status, sem que a consulta a esses dois últimos consuma o turno.

4.3 Criação de Personagem e Progressão

O jogo possui um sistema de criação de personagem no qual o usuário escolhe um nickname e uma raça, entre Humano, Elfo, Anão, Goblin e Draconato. Cada raça possui atributos distintos de vida, defesa, velocidade e mana, além de uma capacidade máxima de peso de inventário e um bônus passivo próprio, aplicado automaticamente durante a partida:

- Humano – não possui bônus especial;

- Elfo – causa dano adicional ao atacar com armas de mana;
- Anão – recebe 5% a menos de dano em combate;
- Goblin – ganha 25% a mais de experiência ao derrotar inimigos;
- Draconato – regenera 1 ponto de vida a cada fase.

A progressão do jogador ocorre por meio do avanço nas fases: a cada ação concluída, ele avança para a fase seguinte, totalizando cinquenta e uma fases até o confronto final. Ao acumular cem pontos de experiência, o personagem sobe de nível, ganhando pontos adicionais de vida máxima.

4.4 Sistemas de Combate, Inventário e Sobrevivência

O combate ocorre por turnos. A cada turno, o jogador pode atacar, tentar fugir ou usar um item do inventário. O dano de cada ataque é calculado a partir do ataque de quem golpeia e da defesa do alvo, com uma pequena variação aleatória e uma chance de acerto crítico. Antes de calcular o dano, o jogo verifica se o golpe realmente acerta: a chance de acerto depende da diferença de velocidade entre atacante e alvo, de modo que personagens mais rápidos erram menos e acertam mais.

Durante o combate, o jogador pode sofrer efeitos de status como veneno, fogo e doença, cada um causando dano ao longo de várias rodadas; a doença, especificamente, transfere parte da vida perdida pelo jogador para o monstro. Alguns monstros também possuem a habilidade de roubar vida diretamente do jogador a cada ataque, e alguns chefes (bosses) impedem que o jogador fuja do combate.

O jogador pode equipar uma arma e diversas peças de armadura simultaneamente (capacete, peitoral, pernas, botas, escudo e um acessório do tipo anel), cada uma ocupando um espaço próprio e somando sua defesa ao total do personagem. Armas do tipo "arma de mana" consomem mana a cada ataque em troca de causar mais dano.

Há também um sistema de fome, que diminui a cada fase avançada e a cada turno de combate; caso o jogador tente fugir, a fome também é consumida, mesmo que a fuga não seja bem-sucedida. Quando a fome chega a zero, o personagem passa a perder vida. Os recursos do jogo são obtidos por meio de drops de monstros derrotados, compras em lojas, achados em baús ou fabricação (crafting). Todos os itens ficam disponíveis no inventário, que possui um limite de peso definido pela raça

do personagem: ao ultrapassá-lo, o jogador fica sobrecarregado, sofrendo penalidades de velocidade e de ataque em combate.

Os inimigos evoluem seus atributos conforme o avanço das fases: quanto mais avançada a fase, maior costuma ser a vida e o poder dos oponentes. Em algumas fases, há duas possibilidades de inimigos ou de eventos, sorteados aleatoriamente. A condição de derrota depende da vida do jogador chegar a zero; nesse caso, é necessário reiniciar o jogo. Ao derrotar um inimigo, o jogador recebe experiência (XP) e ouro, além dos itens de drop garantidos por aquele monstro, quando houver.

4.5 Estrutura de Fases e Narrativa

O jogo é dividido em quatro atos narrativos:

- Ato 1 – Floresta (fases 1 a 11);
- Ato 2 – Minas (fases 12 a 24);
- Ato 3 – Ruínas Arcanas (fases 25 a 37);
- Ato 4 – Castelo (fases 38 a 51), que inclui o confronto final contra o Lorde do Loop, dividido em duas formas de combate consecutivas.

Cada fase pode conter combates, eventos narrativos, escolhas, armadilhas, baús, fontes de cura, vendedores, minigames e momentos de fabricação de itens, reforçando o tema central de loop temporal que dá nome ao jogo.

5 PROGRAMAÇÃO

5.1 Organização do código

O código foi desenvolvido em um único arquivo Python, com aproximadamente três mil duzentas e sessenta linhas, organizadas em cerca de sessenta e cinco funções responsáveis por tarefas específicas, exibição de menus, criação de personagem, gerenciamento de inventário, sistema de combate e execução dos eventos de cada fase. Embora a divisão em múltiplos arquivos pudesse facilitar a manutenção, optou-se por manter o projeto em um único arquivo devido ao tempo disponível e ao nível de conhecimento sobre organização modular no momento do desenvolvimento.

5.2 Entrada de comandos e validação

A interação do jogador ocorre principalmente por meio de entradas de texto no terminal. O programa interpreta comandos como /start, /help, /inv, /sts e /renick, além das opções apresentadas durante o combate e dentro do inventário. Foram utilizadas estruturas condicionais e laços de repetição para validar as entradas do usuário e solicitar novas informações sempre que um valor inválido é fornecido.

5.3 Criação de personagem e atributos

A criação do personagem permite escolher um nickname e uma raça, seja digitando o número correspondente, seja digitando o nome da raça. Cada raça possui atributos próprios (vida, defesa, velocidade, mana, capacidade de peso e um bônus passivo), armazenados em estruturas de dados e utilizados ao longo de toda a partida.

5.4 Inventário, equipamento e crafting

O inventário é armazenado em um dicionário, associando cada item à sua quantidade, e é acessado por meio de um menu numerado, com oito ações principais: usar/consumir, fabricar, descartar, ordenar/filtrar, ver detalhes, equipar/desequipar, buscar por tipo e sair. Cada item recebe um identificador numérico dentro do menu, o que permite selecioná-lo diretamente.

Os itens são organizados por categorias (armas, armaduras/acessórios, consumíveis, alimentos, recursos, drops e itens especiais) e cada um possui peso, valor em ouro e uma descrição própria, consultável na tela de detalhes. O sistema de equipamento permite usar uma arma e até seis peças de armadura/acessórios ao mesmo tempo (capacete, peitoral, pernas, botas, escudo e anel), cada uma somando sua defesa ao total do personagem; equipar uma arma soma o dano dela ao ataque base do jogador, nunca o reduzindo.

O sistema de fabricação (crafting) verifica, para cada item craftável, se o jogador possui os materiais necessários, indicando essa condição diretamente no menu. Alguns itens podem ser fabricados na mão, em qualquer momento; outros exigem a forja de um dos vendedores do jogo.

5.5 Sistema de combate e sobrevivência

O combate é resolvido por turnos, alternando entre as ações do jogador e do monstro. Durante o turno do jogador, é possível atacar, fugir, usar um item ou consultar status e inventário sem custo de turno. O dano de cada ataque considera o ataque de quem golpeia, a defesa de quem apanha, uma margem de variação aleatória e uma chance de acerto crítico. Também foi implementada uma chance de errar completamente o golpe, calculada a partir da diferença de velocidade entre atacante e alvo.

O sistema controla os valores de vida do jogador e do monstro, determinando vitória ou derrota a partir deles. Além disso, o jogo possui um sistema de fome, que se altera a cada ação, e efeitos negativos de status (veneno, fogo e doença), aplicados por armadilhas ou por ataques de determinados monstros, causando dano ao longo de várias rodadas.

5.6 Progressão, inimigos e aleatoriedade

O jogo possui cinquenta e uma fases distribuídas em quatro atos narrativos. Os inimigos apresentam evolução de atributos conforme o avanço das fases, e alguns bosses específicos impedem que o jogador fuja do combate. Em diversas situações, o programa utiliza geração de valores aleatórios para sortear eventos, inimigos e resultados de ataques, aumentando a variação entre partidas. A progressão também envolve experiência (XP), níveis, ouro e bônus permanentes obtidos por meio de decisões tomadas durante a história.

5.7 Recursos utilizados na interface

Para melhorar a apresentação no terminal, foram utilizados códigos ANSI para aplicação de cores e a biblioteca `time` para controlar pausas e para produzir um efeito de escrita lenta, exibindo o texto das fases caractere por caractere, como se estivesse sendo digitado em tempo real. Também foram implementados recursos de limpeza do terminal e de organização das mensagens apresentadas ao jogador, incluindo pausas explícitas para leitura antes de a tela ser limpa novamente.

5.8 Principais conceitos de Python aplicados

Durante o desenvolvimento, foram aplicados conceitos estudados em programação, incluindo variáveis, estruturas condicionais (`if/elif/else`), laços de

repetição (for e while), funções, listas, dicionários e o uso de bibliotecas padrão da linguagem. A aplicação desses conceitos permitiu transformar as regras do jogo em sistemas interativos e conectados entre si.

6 DIFICULDADES ENCONTRADAS

O prazo disponível para o desenvolvimento não foi suficientemente longo, o que impediu a implementação de todas as funcionalidades planejadas inicialmente. Além disso, para enriquecer a experiência do jogador, foram utilizadas estruturas ainda não estudadas em aula, como dicionários, códigos ANSI, a biblioteca time e a geração de números aleatórios. Isso gerou dificuldades significativas, pois foi necessário aprender esses recursos por conta própria.

A criação do sistema de combate entre monstro e jogador também representou um grande desafio, por envolver diversos valores e funções interdependentes. A organização do código tornou-se complexa à medida que o arquivo único crescia, chegando a mais de três mil linhas. Embora tenha sido cogitada a divisão em múltiplos arquivos, optou-se por mantê-lo monolítico, em função do tempo necessário para reestruturar e conectar os módulos.

7 SOLUÇÕES ADOTADAS

Para superar as dificuldades, foram realizadas pesquisas no Google, no YouTube e em explicações de ferramentas de inteligência artificial. A inteligência artificial foi utilizada especificamente para a implementação dos códigos ANSI e da biblioteca time, devido à falta de conhecimento prévio sobre esses recursos; mesmo após a consulta a tutoriais em vídeo, não foi possível aplicar esses conteúdos de forma totalmente autônoma. A IA também auxiliou na correção ortográfica e gramatical dos textos das fases do jogo. Toda a história e a narrativa foram escritas manualmente pelos autores; apenas a revisão linguística contou com apoio de ferramentas de inteligência artificial (Grok, Gemini e Claude).

8 RESULTADOS

O resultado final agradou aos autores e manteve-se fiel às ideias originais do projeto, embora diversas adaptações tenham sido realizadas ao longo do desenvolvimento para viabilizar sua conclusão. Considera-se que o jogo ficou completo e, principalmente, divertido para o jogador.

8.1 Funcionalidades Implementadas

O jogo The Infinite Loop (versão 1.0) é um RPG de texto medieval completo, desenvolvido em Python puro, que conta com as seguintes funcionalidades principais:

- Sistema de menu e comandos globais, com validação de entradas inválidas;
- Criação de personagem com escolha de nickname e raça (Humano, Elfo, Anão, Goblin e Draconato), cada uma com atributos, capacidade de carga e bônus passivo distintos;
- Sistema completo de status (vida, mana, fome, ouro, XP, nível, ataque, armadura, defesa, velocidade e peso do inventário);
- Inventário baseado em dicionário, acessado por um menu numerado com oito ações (usar, fabricar, descartar, ordenar, ver detalhes, equipar/desequipar, buscar e sair), com itens organizados por categoria e descrições individuais;
- Sistema de equipamento com uma arma e seis peças de armadura/acessórios simultâneas (capacete, peitoral, pernas, botas, escudo e anel), cada uma somando sua defesa ao total do personagem;
- Sistema de peso e sobrecarga, com capacidade máxima variando por raça e penalidades de velocidade e ataque quando excedida;
- Sistema de crafting (fabricação na mão ou na forja) com verificação automática de materiais disponíveis;
- Combate por turnos com ataque, fuga e uso de itens, incluindo chance de acerto crítico, chance de errar o golpe (baseada em velocidade), status negativos (veneno, fogo e doença), monstros que roubam vida e chefes que impedem a fuga;
- Armas de mana, que consomem mana por ataque em troca de dano adicional;
- Cinquenta e uma fases organizadas em quatro atos narrativos, com eventos, escolhas, armadilhas, baús, fontes de cura, lojas e minigames;

- Quatro vendedores distintos (Otto, Gol, Vivian e Othon), cada um com um estoque próprio de itens, incluindo armas, armaduras, poções e acessórios;
- Progressão por XP e nível, com aumento de vida máxima a cada nível, e tela de estatísticas ao final da sessão;
- Recursos de qualidade de vida, como cores ANSI multiplataforma, efeito de escrita lenta na narrativa, limpeza inteligente de tela, pausas para leitura e validação de entradas.

8.2 Pontos de Melhoria

Acredita-se que o sistema de combate ainda poderia ser tornado mais intuitivo e divertido, com mais opções de ações e habilidades selecionáveis pelo jogador. A exibição e a decoração da saída do terminal poderiam ser aprimoradas com tabelas e elementos visuais mais elaborados. Também seria interessante ampliar o número de itens, receitas de fabricação e tipos de inimigos disponíveis. Por fim, alguns bugs remanescentes poderiam ser corrigidos em versões futuras.

9 CONCLUSÃO

Ao longo do projeto, foram aplicadas todas as funcionalidades básicas ensinadas em aula, além de terem sido desenvolvidas habilidades adicionais de forma prática e intuitiva. O resultado final superou as expectativas dos autores, que trabalharam no projeto desde o início das férias e se sentem orgulhosos do jogo produzido. Eles nunca haviam imaginado conseguir desenvolver algo dessa complexidade. Até o momento, esta foi a experiência mais divertida com a linguagem Python. Espera-se, no futuro, poder aprimorar o código com novos conhecimentos adquiridos ao longo do curso técnico.

REFERÊNCIAS

APOENA STACK. Tutoriais de programação. YouTube. Disponível em: <https://youtu.be/yfAixrxCVfo>. Acesso em: 15 ago. 2026.

CURSO EM VÍDEO. Python 3 – Mundo 1. YouTube. Disponível em: <https://youtu.be/0hBIhkcA8O8>. Acesso em: 15 ago. 2026.

CURSO EM VÍDEO. Python 3 – Mundo 2. YouTube. Disponível em: <https://youtu.be/ZWj8o692qGY>. Acesso em: 15 ago. 2026.

PYTHON SOFTWARE FOUNDATION. Python.org. Disponível em: <https://www.python.org/>. Acesso em: 15 ago. 2026.

Nota: para esclarecimento de dúvidas pontuais e apoio na implementação de trechos específicos do código, foram utilizadas as ferramentas de inteligência artificial Grok, Gemini e Claude.